TeachForge — Handoff para otra IA y guía paso a paso

Documento autocontenido. Pégalo completo en la otra IA (Freebuff u otra) para seguir construyendo sin perder el contexto de la conversación con Claude.

0. Prompt de arranque (pégalo primero)

Eres mi copiloto de código para un hackatón. Lee TODO este documento antes de actuar. Vamos a construir TeachForge siguiendo la sección 7, un paso a la vez.

Reglas:

1. Haz un solo paso por turno y espera mi resultado antes de seguir.
2. No añadas funciones fuera de alcance ni cambies el stack sin avisarme.
3. Al terminar cada paso, dime cómo probarlo y cuál es el criterio de aceptación.
4. Si algo falla, da primero un diagnóstico mínimo y luego el arreglo.
5. Respeta las reglas de honestidad de la sección 9.
6. Commits pequeños y convencionales (feat, fix, docs).
7. Una demo desplegada y estable vale más que una función extra.
8. Empieza preguntándome solo lo imprescindible: qué proveedor de LLM usaré y qué cuentas ya tengo.

1. Contexto del hackatón

- **Evento:** DSC HACK 2026 (PUCP). Jueves 29 y viernes 30 de octubre, remoto. Sábado 31 presencial en NTT DATA.
- **Reto:** «Educación 2036 — las grietas que ya se ven». Elegir una grieta educativa del Perú de hoy con dato y fuente, argumentar por qué crecerá hacia 2036 y construir una solución de software desplegada, probada por al menos una persona que vive el problema.
- **Mi grieta:** 03, docentes sin relevo. Dato de la slide: 22% de los docentes de secundaria tiene 55 años o más (OCDE Education at a Glance 2026 y GRADE vía Gestión, 19/10/2026; Minedu vía La República, 05/07/2026). Ese dato mide edad, no jubilaciones; hay que verificarlo en la fuente original.
- **Rúbrica (un jurado por criterio):** Problemática 30% (la grieta, su evidencia y lo que aprendí del usuario); Innovación 30% (mi lectura de hacia dónde crece y qué tan poco obvia es la respuesta); Técnico y factibilidad 20% (que funcione, esté desplegado y

pueda sostenerse); Pitch y caso de negocio 20% (que se entienda y alguien pueda adoptarlo).

- **Lo que no alcanza:** chatbot genérico de profe u orientador, detector de textos de IA, bolsa de trabajo sin modelo distinto, «ChatGPT con otro nombre».
- **Entregables (formulario, cierra el sábado 31 a las 17:00, después no se acepta nada):** repo público de GitHub, URL del pitch, demo desplegado y funcionando, carpeta de evidencias (hasta 3 registros con la plantilla oficial; al menos 1 es prueba de uso del producto, no una charla sobre el problema).
- **Sábado:** llegada 9:00 con DNI físico y laptop, checkpoint técnico 10:00, almuerzo 13:00, checkpoint de producto 15:00, cierre 17:00, premiación 19:15. Meta: enviar todo hacia las 16:00.

2. El proyecto

TeachForge convierte la experiencia pedagógica de docentes veteranos en casos reutilizables y los usa para entrenar a docentes nuevos en simulaciones de aula.

Tesis: el criterio pedagógico de un docente no debería jubilarse con él.

Lectura no obvia: lo que se pierde al jubilarse un docente no es contenido (está en libros y en la IA) sino conocimiento pedagógico del contenido: qué error comete el alumno, por qué, y qué explicación lo destraba. Con IA el contenido es casi gratis y el criterio es lo escaso.

Ciclo del MVP: Captura (voz o texto) → caso estructurado que el veterano revisa y valida → biblioteca de casos → simulación de clase de 3 turnos con 3 estudiantes simulados → evaluación por rúbrica con citas textuales → perfil de competencias y siguiente práctica.

Toque 2036 opcional: «momento IA»: un estudiante llega con una respuesta que le dio una IA, con un error plausible, y el docente debe notarlo.

3. Decisiones tomadas

- Versión simple y desplegada primero; lo avanzado entra solo si el núcleo ya funciona (puertas en la sección 7).
- **Captura por voz incluida** (a muchos veteranos les da flojera escribir), con transcripción siempre editable y texto como respaldo.
- **Base de conocimiento simulada en PostgreSQL** (Supabase) para la demo: casos de ejemplo marcados is_seed = true y visibles como «Ejemplo» en la app. El caso real es el del docente que capturemos en vivo.
- **RAG con pgvector** para un buscador de experiencias (fase posterior).

- **Grafo con Neo4j** (Aura Free) como fase final, con respaldo desde Postgres.
- Simulación simple: una llamada por turno que interpreta 3 estudiantes con un solo prompt.
- Perfil de competencias ligero (versión mínima de un digital twin), no un clon del docente.
- Recomendación con regla simple: la competencia más baja manda la siguiente práctica.
- El veterano valida su propio caso (revisión humana).
- No se usa un sistema multiagente ni se afirma que existe; son tres roles de IA con prompts distintos: extractor, estudiantes simulados y evaluador.
- Competencias: 1 Diagnóstico del error, 2 Estrategia de explicación, 3 Verificación de la comprensión, 4 Adaptación a diferencias, 5 Uso crítico de la IA (solo con momento IA).

4. Stack, variables y estructura del repo

Stack: Next.js (App Router, TypeScript), Tailwind, Zod, Supabase (Postgres + pgvector), Vercel. SDK del proveedor de LLM aislado en lib/llm.ts. Para gráficos: Recharts; para el grafo: react-force-graph-2d o similar. Plan B si hay retraso: Streamlit + Supabase.

Proveedor de IA: elegir UNO que cubra chat, transcripción de audio y embeddings si es posible (verificar en su documentación); si no, usar uno para chat y otro para audio y embeddings. Modelo más capaz para extracción y evaluación; uno más barato y rápido para estudiantes simulados.

Variables de entorno (.env.example):

```
LLM_API_KEY=
TRANSCRIBE_API_KEY=
EMBEDDINGS_API_KEY=
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_KEY=
NEO4J_URI=
NEO4J_USER=
NEO4J_PASSWORD=
DEMO_MODE=false
```

Estructura:

```
teachforge/
├─ README.md, LICENSE (MIT), .env.example, .gitignore
├─ docs/ (problema-y-evidencia, arquitectura, rubrica, pitch, declaracion-de-
ia-y-trabajo-previo)
├─ prompts/ (extract_case.md, simulate_students.md, evaluate_attempt.md,
generate_seed_cases.md)
├─ supabase/ (schema.sql)
├─ fixtures/ (seed_cases.json, respuestas guardadas para DEMO_MODE)
├─ scripts/ (seed.ts, load_graph.ts)
├─ src/app/ (page, capture, cases, simulate/[caseId], profile, graph, api/*)
├─ src/lib/ (llm.ts, db.ts, schemas.ts, rubric.ts, cost.ts, graph.ts)
├─ src/components/ (Recorder, CaseReview, Chat, ScoreBars)
└─ tests/
```

5. Datos y contratos de API

schema.sql:

```
create extension if not exists pgcrypto;
create extension if not exists vector;

create table cases (
  id uuid primary key default gen_random_uuid(),
  created_at timestamptz default now(),
  author_alias text not null,
  years_experience int,
  subject text not null,
  level text not null,
  topic text not null,
  raw_story text not null,
  structured jsonb not null,
  tags text[] default '{}',
  validated_by_author boolean default false,
  is_seed boolean default false,
  consent_at timestamptz,
  embedding vector(1536)
);

create table attempts (
  id uuid primary key default gen_random_uuid(),
  created_at timestamptz default now(),
  case_id uuid references cases(id) on delete cascade,
  learner_alias text not null,
  transcript jsonb not null,
  hidden_states jsonb,
```

```
  tokens_in int, tokens_out int, cost_usd numeric
);

create table evaluations (
  id uuid primary key default gen_random_uuid(),
  created_at timestamptz default now(),
  attempt_id uuid references attempts(id) on delete cascade,
  scores jsonb not null,
  summary text,
  next_focus text,
  human_reviewed boolean default false
);

create or replace function match_cases(query_embedding vector(1536),
match_count int default 5)
returns table (id uuid, topic text, similarity float)
language sql stable as $$
  select id, topic, 1 - (embedding <=> query_embedding) as similarity
  from cases where embedding is not null
  order by embedding <=> query_embedding
  limit match_count;
$$;
```

La dimensión 1536 debe coincidir con el modelo de embeddings elegido. Privacidad: alias, sin datos personales, sin login para el MVP, audio no guardado.

Caso estructurado (Zod): titulo, materia, nivel, tema, concepto_clave; error_frecuente { descripcion, ejemplo_alumno, por_que_ocurre }; estrategia_que_funciono { descripcion, pasos[], condiciones }; lo_que_no_funciono; senal_de_comprension; contexto_aula; perfiles_estudiantes[] { nombre, perfil, como_se_manifiesta }; campos_faltantes[] (preguntas para el docente).

Rutas:

- POST /api/transcribe: multipart con campo audio → { text } o { error }.
- POST /api/extract: { story, meta } → { structured, missing[] }.
- POST /api/cases: { author_alias, years_experience, subject, level, topic, raw_story, structured, consent } → { id }.
- GET /api/cases → lista con id, title, topic, is_seed, author_alias, tags.
- GET /api/search?q= → casos más similares con similarity (RAG).
- POST /api/simulate: { caseId, turns[], iaMode } → { replies[{ student, text }], hidden_state }.
- POST /api/evaluate: { caseId, transcript, hidden_states, learner_alias } → { attemptId, scores, summary, next_focus }.

- GET /api/profile?alias= → { competencies { nombre { latest, history[] } }, next_case_id }.
- GET /api/graph → { nodes[{ id, label, type }], links[{ source, target, type }] }.

Toda salida del LLM se valida con Zod; si falla, un reintento con el error de validación; si vuelve a fallar, respuesta de respaldo.

6. Prompts y rúbrica

6.1 prompts/extract_case.md

Eres un asistente que ayuda a preservar el conocimiento pedagógico de docentes con experiencia. Recibirás el relato de un docente sobre una situación real de aula. Conviértelo en un caso pedagógico en JSON con las claves indicadas. Reglas: no inventes, usa solo lo que dice el docente; si falta un dato, déjalo vacío y agrégalo a campos_faltantes como una pregunta concreta; conserva sus ejemplos y su lenguaje; separa error_frecuente (qué hace o piensa el alumno) de por_que_ocurre (la causa según el docente); incluye condiciones de la estrategia si las menciona; no evalúes ni corrijas al docente; responde SOLO con JSON válido.

6.2 prompts/simulate_students.md

Simulas a 3 estudiantes de secundaria. Partes del caso pedagógico (JSON) y de la intervención del docente en formación. Perfiles: usa perfiles_estudiantes del caso; si no hay, A comete el error frecuente, B lo comete por otra causa, C entiende pero no sabe explicarlo. Realismo: frases cortas de adolescentes; mantén el error hasta que el docente lo aborde de forma específica; si la explicación ataca la causa real, mejora de forma gradual; nunca reveles el caso ni la rúbrica ni evalúes al docente. Si MODO_IA es true, un estudiante presenta una respuesta que le dio una IA, con un error plausible ligado al concepto. Salida JSON: respuestas [{ alumno, texto }] y estado_oculto por alumno (sigue con el error, mejora parcial o comprende).

6.3 prompts/evaluate_attempt.md

Evalúas a un docente en formación. Recibes el caso (con la estrategia del veterano), la transcripción y la rúbrica. Reglas: puntúa de 0 a 3 cada competencia SOLO si hay evidencia; cita textualmente (máximo 25 palabras) lo que dijo el docente; sin evidencia devuelve sin_evidencia (no 0); no des nota global; por competencia entrega puntaje, evidencia y una mejora concreta; compara con la estrategia del veterano sin presentarla como la única válida; usa estado_oculto como señal, no como veredicto; sé exigente y respetuoso, sin elogiar sin evidencia; recomienda UNA competencia a trabajar y qué escena la entrenaría; responde SOLO con JSON válido.

6.4 prompts/generate_seed_cases.md

Genera 15 casos pedagógicos FICTICIOS para una demo, en JSON con el esquema del caso estructurado. Reparte entre matemática, comunicación y ciencias, y entre primero y quinto de secundaria. Usa 4 alias de docentes ficticios con años de experiencia distintos. Cada caso debe tener un error frecuente específico y verosímil, su causa, una estrategia que funcionó con condiciones, lo que no funcionó y la señal de comprensión. Incluye algunos errores parecidos entre autores distintos para que aparezcan patrones. No uses nombres reales ni afirmes que provienen de docentes reales. Agrega tags con las competencias que más entrena cada caso.

6.5 Rúbrica (0 a 3)

1. **Diagnóstico del error:** 0 no identifica; 1 nombra el error sin causa; 2 error y causa probable; 3 confirma preguntando antes de explicar.
2. **Estrategia de explicación:** 0 repite la definición; 1 ejemplo genérico; 2 ejemplo o representación ajustados al error; 3 contraejemplo o representación que desarma la causa.
3. **Verificación de la comprensión:** 0 pregunta «¿entendieron?»; 1 ejercicio igual al ejemplo; 2 pregunta que distingue memorizar de entender; 3 pide aplicar en un caso nuevo y ajusta con la respuesta.
4. **Adaptación a diferencias:** 0 una sola vía; 1 reconoce diferencias y no actúa; 2 ajusta para un perfil; 3 rutas distintas para los tres perfiles.
5. **Uso crítico de la IA (solo con momento IA):** 0 no detecta el error; 1 lo detecta sin explicar; 2 lo detecta y explica; 3 lo convierte en aprendizaje para la clase.

Las anclas deben validarse con al menos un docente real.

7. Paso a paso

Tiempos aproximados. Duración total del núcleo (S0 a S10): unas 12 a 14 horas; lo demás es opcional y depende de las puertas.

S0. Cuentas y claves (30 min). GitHub, Vercel, Supabase, clave del LLM (más transcripción y embeddings), Neo4j Aura solo si llegas a S13. *Aceptación:* una llamada de prueba al LLM responde desde mi máquina.

S1. Repo y hola mundo desplegado (45 min).

```
npx create-next-app@latest teachforge --typescript --tailwind --app
cd teachforge
npm i zod @supabase/supabase-js recharts
git init && git add . && git commit -m 'chore: init TeachForge'
gh repo create teachforge --public --source=. --push
```

Conectar el repo a Vercel. *Aceptación:* la URL pública abre; repo público con LICENSE MIT y README mínimo.

S2. Base de datos (30 min). Aplicar schema.sql en Supabase, crear lib/db.ts, cargar variables en Vercel. *Aceptación:* una ruta /api/health inserta y lee un caso.

S3. Núcleo (45 min). lib/llm.ts con una función que recibe system, user y un esquema Zod, reintenta si falla la validación y devuelve tokens usados; lib/schemas.ts, lib/rubric.ts, lib/cost.ts; carga de prompts desde /prompts. *Aceptación:* un test valida un JSON correcto y rechaza uno inválido.

S4. Captura en texto (2 h). Página /capture con las 6 preguntas guiadas (tema y nivel; error repetido; cómo lo noté; qué funcionó y en qué condiciones; qué no funcionó; cómo supe que entendieron) y un cuadro de relato; POST /api/extract; pantalla de revisión editable con campos faltantes resaltados; casilla de consentimiento; POST /api/cases. *Aceptación:* pego un relato, veo el caso estructurado, lo edito, lo valido y queda en Supabase con validated_by_author = true.

S5. Voz (1,5 a 2 h). Componente Recorder con MediaRecorder (audio/webm;codecs=opus, audioBitsPerSecond 32000; en Safari audio/mp4), botones grabar y detener y temporizador; POST /api/transcribe con FormData (Vercel limita el cuerpo a unos 4,5 MB: verificar el límite actual); el texto transcrito llena el cuadro de relato y es editable; no guardar el audio. *Aceptación:* grabo 2 a 3 minutos en el celular y en Chrome de escritorio y obtengo texto editable; si falla, aparece un mensaje claro y puedo escribir.

S6. Biblioteca de casos (45 min). /cases lista los casos con etiqueta Real o Ejemplo, alias del autor y tema, con botón Practicar. *Aceptación:* se distinguen a simple vista los reales de los de ejemplo.

S7. Simulador (2 h). /simulate/[caseId] con escena, perfiles, chat de 3 turnos y toggle Momento IA; POST /api/simulate con el prompt 6.2; guardar transcript y estado_oculto. *Aceptación:* 3 turnos completos; los estudiantes mantienen el error ante una explicación genérica y mejoran gradualmente si se ataca la causa.

S8. Evaluación (1,5 h). POST /api/evaluate con el prompt 6.3; mostrar puntajes con citas; guardar en evaluations; botón «Ver cómo lo hizo la docente» que revela la estrategia del caso. *Aceptación:* cada puntaje cita texto del usuario; sin evidencia se muestra «sin evidencia».

PUERTA A: si el ciclo S4 a S8 funciona y está desplegado, seguir. Si no, parar y arreglarlo antes de añadir nada.

S9. Perfil (1 h). /profile con barras por competencia (último puntaje y evolución entre intentos) y siguiente práctica por regla simple: la competencia más baja manda un caso con esa etiqueta; si no hay, el más reciente. *Aceptación:* dos intentos del mismo alias muestran evolución.

S10. Endurecer (1,5 h). Reintento con validación Zod, DEMO_MODE con fixtures, estados de carga y error, límite básico de uso, registro y vista mínima del costo por sesión, responsive básico. *Aceptación:* con la API caída y DEMO_MODE=true la demo completa sigue funcionando.

PUERTA B: si quedan horas reales (después de reservar tiempo para evidencia, sueño, llegada del sábado y ensayo del pitch), seguir con S11 a S13 en ese orden.

S11. Base simulada (1 a 1,5 h). Generar 15 casos con el prompt 6.4, revisarlos a mano, guardarlos en fixtures/seed_cases.json; scripts/seed.ts los inserta con is_seed = true y tags. *Aceptación:* 15 casos visibles, todos con etiqueta Ejemplo.

S12. RAG (1,5 a 2 h). Crear embeddings al guardar un caso y en el seed; función match_cases; GET /api/search y un buscador de experiencias en /cases. *Aceptación:* buscar «signos en ecuaciones» devuelve casos relevantes primero.

S13. Grafo (3 a 4 h, solo si todo lo anterior está estable). Nodos: Caso, Error, Estrategia, Concepto. Opción A, Neo4j Aura Free con neo4j-driver (URI neo4j+s) y un script de carga. Opción B, tabla de aristas en Postgres. Cypher de carga:

```
MERGE (c:Caso {id: $id}) SET c.titulo = $titulo, c.is_seed = $seed
MERGE (e:Error {texto: $error})
MERGE (s:Estrategia {texto: $estrategia})
MERGE (k:Concepto {nombre: $concepto})
MERGE (c)-[:PRESENTA]->(e)
MERGE (e)-[:SE_DESTRABA_CON]->(s)
MERGE (c)-[:ENSENA]->(k)
```

GET /api/graph y página /graph. Verificar que la instancia no se pause antes de la demo y tener respaldo desde Postgres. *Aceptación:* se ve el grafo con al menos un patrón entre casos, con el aviso «datos simulados».

S14. Evidencia (en paralelo, empezar ya). Conseguir un docente veterano y un docente novato. Registro 1: el veterano usa Captura con la app y valida su caso (medir fidelidad de 1 a 5, número de ediciones, minutos, una frase suya). Registro 2: el novato resuelve la simulación y ve su evaluación (puntajes del intento 1 frente al 2, dónde se trabó, qué cambié). Registro 3: entrevista corta a un docente o directivo sobre el relevo. Usar la

plantilla oficial (acceso desde el correo de postulación), consentimiento por escrito, alias, y una línea de «qué cambié por esto». *Aceptación:* 3 registros completos antes del sábado a las 14:00; nada se modifica después del cierre.

S15. Cierre (sábado). README final con URL de la demo, capturas, qué funciona hoy y qué es hoja de ruta; docs/declaracion-de-ia-y-trabajo-previo.md; video de respaldo; probar la demo desde otro dispositivo; enviar el formulario hacia las 16:00. *Aceptación:* formulario enviado y las URLs abren sin sesión.

8. Pitch y evidencia (resumen)

- **Gancho:** «Cada año se jubilan docentes que saben cosas que ningún libro dice. Hoy esas cosas se van con ellos.»
- **Frank Gago (problemática):** dato con su fuente verificada, mecanismo y lo que aprendí del docente real, con una cita.
- **Andre Cruz (innovación):** lo que se pierde es criterio, no contenido; el ciclo convierte experiencia en entrenamiento.
- **José Carlos Guerra (técnico):** demo en vivo, qué funciona hoy y qué es hoja de ruta, fallback y costo por sesión medido.
- **Javier Blanco (negocio):** quién lo adopta (institutos pedagógicos, facultades de Educación, UGEL, redes de colegios), un piloto concreto y límites honestos.
- Hipótesis: H1 (hay conocimiento valioso y capturable) se mide con fidelidad y ediciones; H2 (practicar mejora el segundo intento) solo es indicativa; H3 (perfil frente a certificados) no se afirma.

9. Reglas de honestidad

- Los casos de ejemplo siempre se muestran y se presentan como simulados; nunca como de docentes reales.
- No afirmar multiagente, digital twin completo, grafo o recomendación adaptativa si no están construidos; decirlo como hoja de ruta.
- No afirmar que mejora la calidad docente; solo lo que se midió con pocas personas, como indicativo.
- Los datos de la tabla de evidencia solo se citan si se verificaron en la fuente original.
- Si se reutiliza código de proyectos anteriores, declararlo y preguntar a los organizadores si está permitido.
- Consentimiento y alias en toda evidencia; no guardar audio.

10. Estado actual

- **Hecho:** blueprint del proyecto, maqueta de 3 pantallas (captura y validación, simulador, evaluación y perfil), decisiones de la sección 3.
- **Pendiente:** todo el código, conseguir al docente veterano y al novato, verificar las fuentes del dato de la grieta, crear el repo y desplegar.
- **Prioridad inmediata:** S0 y S1, y pedir ya los docentes para S14.